OUTPUT ANALYSIS

Contact Management System • Circular Doubly Linked List, Array and Queue in C++

1. Time Complexity Analysis

Let n be the number of contact records currently stored in the circular doubly linked list.

Operation	Time Complexity	Reason
Add Contact	**O(n)**	Duplicate-phone check scans the list; finding a free speed-dial slot is a fixed 10-slot scan; insertion at tail (*head->prev*) is O(1).
Delete Contact	**O(n)**	Searching the contact by ID takes O(n); enqueue into the recently-deleted queue (with a dequeue if the queue is full), freeing the slot and unlinking the node are O(1).
Search Contact	**O(n)**	Linear traversal from *head* until the head is reached again; a match on ID or name may need every node.
Display Forward / Backward	**O(n)**	Walks the circular list once using *next* from head or *prev* from tail until it returns to the start, printing each node.
Speed Dial Status	**O(1)**	Scans the fixed-size speed-dial array (MAX_SLOTS = 10), independent of the number of contacts.
Undo Delete (Restore Oldest)	**O(n)**	Peek/dequeue at the queue front is O(1); duplicate-phone check is O(n); re-insertion at tail is O(1).
Delete History	**O(1)**	Reads at most QSIZE = 5 records from front to rear of the queue, so it is bounded by a constant.

2. Space Complexity Analysis

Contact data is stored in dynamically allocated nodes. Each node holds the contact fields (ID, name, phone number, email, speed-dial slot) and two pointers: *prev* and *next*.

Total Space Complexity	**O(n)**	The list holds n contact nodes (fields plus *prev* and *next*); the speed-dial array and queue are fixed-size.
Speed Dial Array	**O(1)**	Always MAX_SLOTS = 10 elements, so it does not grow with the number of contacts.
Deleted Queue	**O(1)**	Circular array queue of QSIZE = 5 records: the size is fixed at compile time and the indices wrap around using modulo.
Auxiliary Space per Operation	**O(1)**	Operations use a few temporary pointers and variables; adding a contact allocates one extra node.

Interpretation: Operations that must locate a contact (add, delete, search, undo) are linear-time because the circular list is searched from *head*. Reaching the tail through *head->prev* makes insertion O(1), and a doubly linked list makes unlinking O(1) once the node is found. The circular queue enqueues and dequeues in O(1) with no shifting, and speed-dial allocation and delete history are bounded by fixed array sizes, so they behave as constant time.